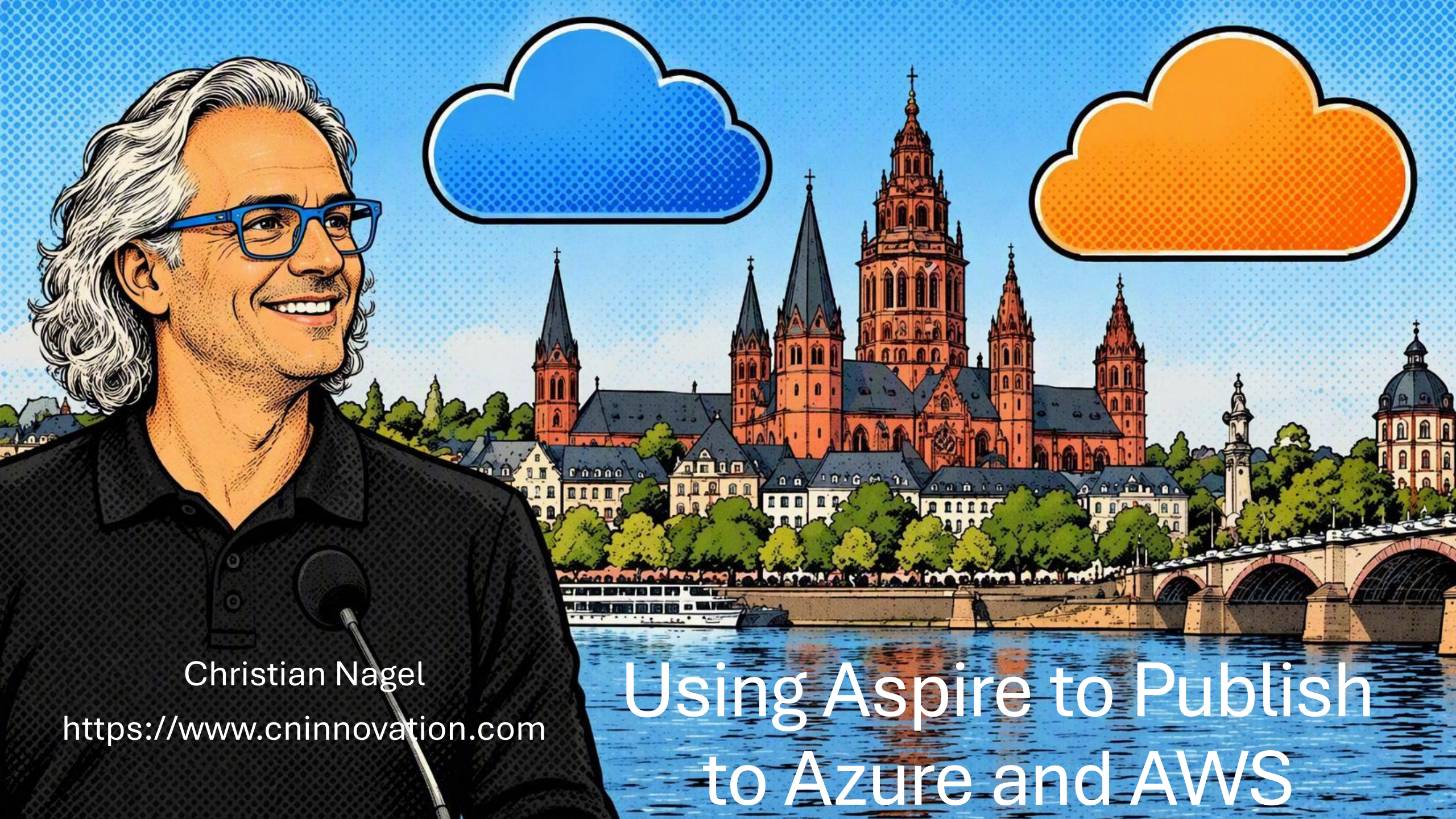




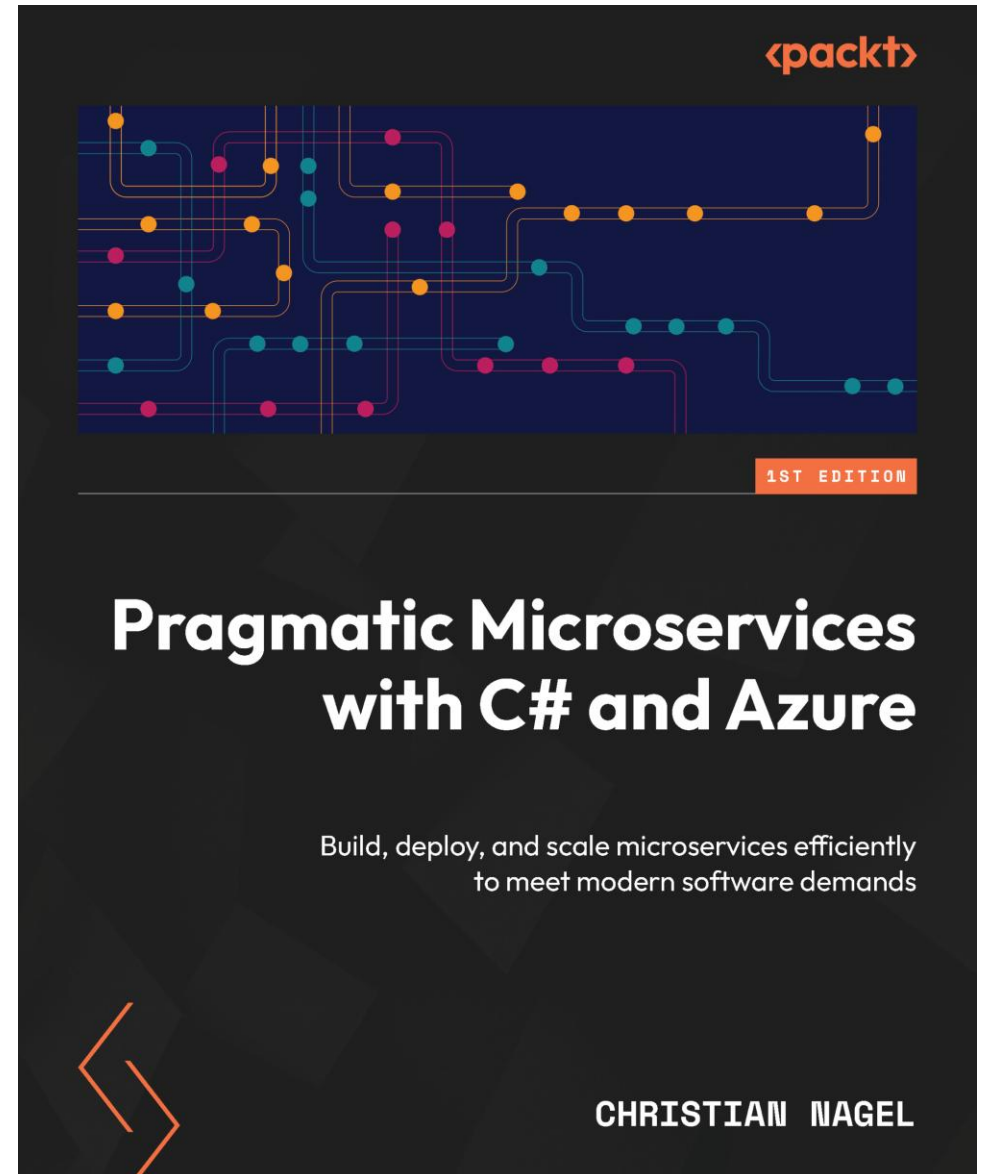
Christian Nagel

<https://www.cninnovation.com>

Using Aspire to Publish to Azure and AWS

Christian Nagel

- Training
 - Coaching
 - Consulting
 - Development
 - New book: **Pragmatic Microservices**
-
- Microsoft MVP
 - www.cninnovation.com
 - csharp.christiannagel.com
 - @christiannagel





What is Aspire?

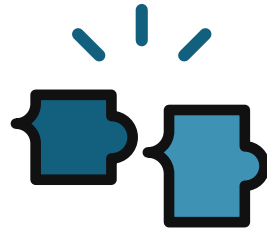
(renamed from .NET Aspire to Aspire)

“Aspire is a code-first orchestration and observability layer for distributed applications. Start with the path that matches your repo, runtime, or workflow.”

<https://aspire.dev/docs/>



Orchestration



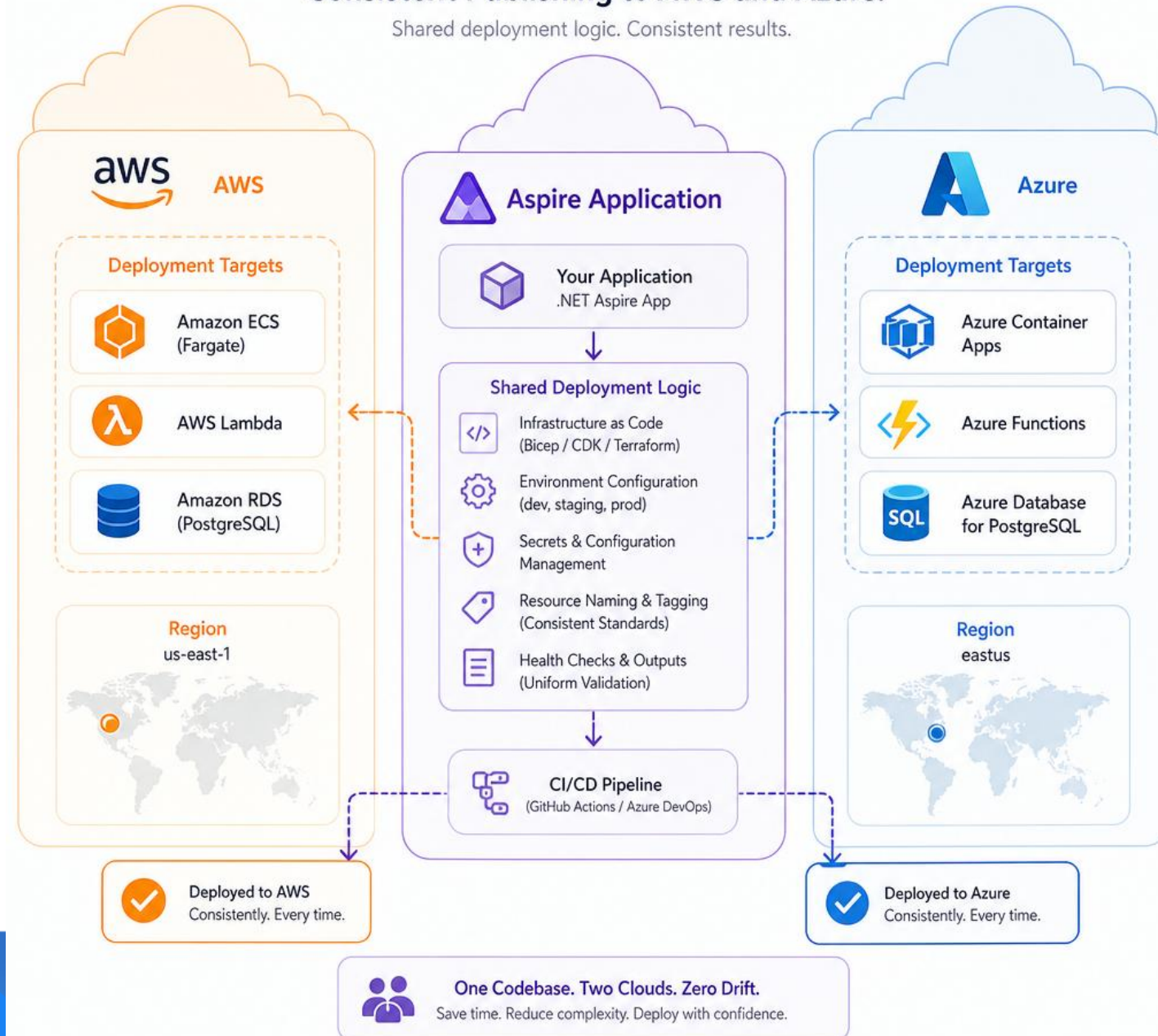
Integrations



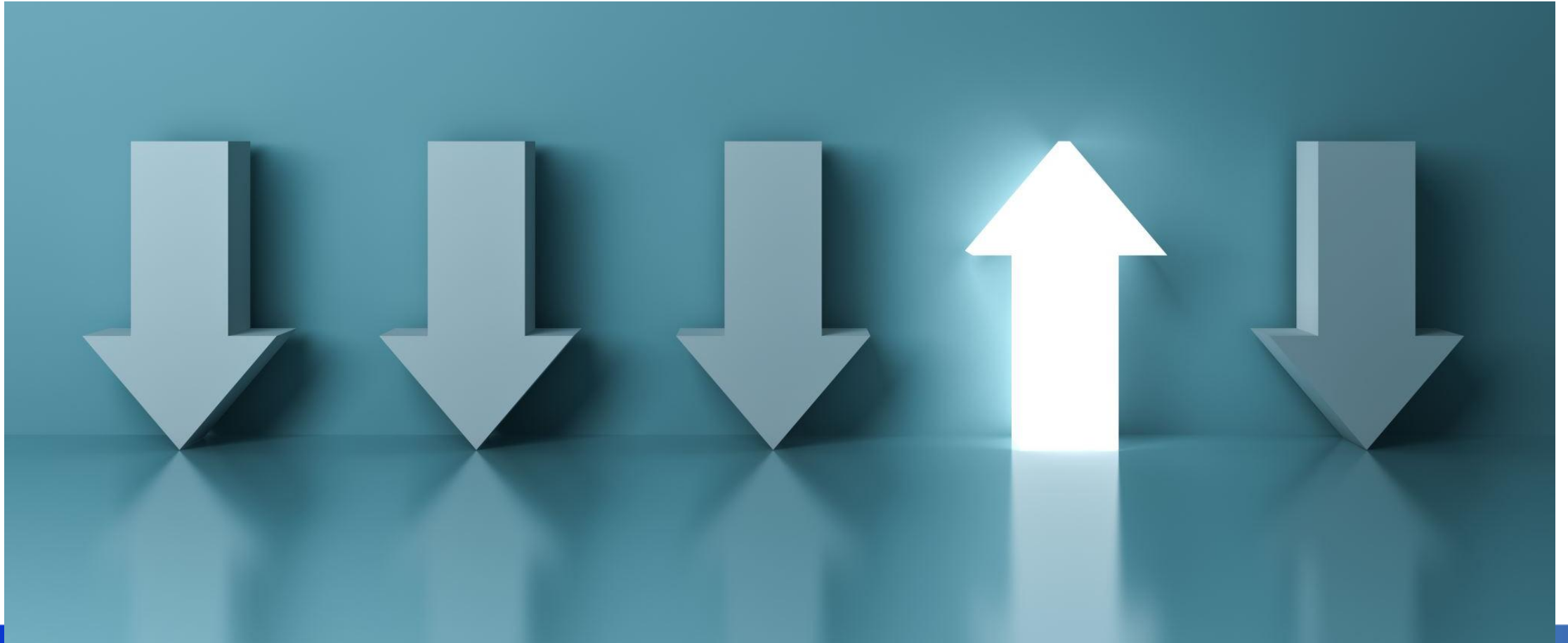
Tooling

Publishing with Aspire to AWS and Azure

One Aspire Application.
Consistent Publishing to AWS and Azure.
Shared deployment logic. Consistent results.



Publish vs Deploy



Aspire 1st Generation

- Create JSON manifest from app-model
- Azure Developer CLI (azd): JSON → Bicep
- aspire CLI: JSON → Docker Compose Files
- aspire CLI: JSON → Kubernetes Helm Charts

Issue:
Updates

Aspire 2nd Generation

- **No more JSON manifest**
 - The generated manifest file is obsolete as the deployment contract
- **Environments define the target**
 - An environment describes where and how resources are created
 - Infrastructure is expressed in C# in the AppHost – not in generated JSON
- **aspire publish**
 - Produces deployment artifacts for the selected environment
- **aspire deploy**
 - Creates and updates the resources directly in the target environment
- **Benefits**
 - One model from local run to cloud deployment, strongly typed and extensible

Publish: Generate the Deployment Artifacts

- **aspire publish** produces artifacts from the AppHost model — no cloud call required
- **Kubernetes:** manifests/Helm charts for pods, services, secrets, and config maps
- **Docker Compose:** a compose file with containers, networks, volumes, and env values
- **Container images:** built and tagged as part of the publish step
- **Reviewable:** artifacts land in source control and pass through normal code review

Deploy: Provision and Push to a Cloud

- **aspire deploy** takes the model all the way to a running environment
- **Azure Container Apps:** provisioned environment, ingress, scaling, and managed identity
- **Azure App Service:** app plans and settings wired from the Aspire resource graph
- **AWS ECS/Fargate:** task definitions, services, and registry push via the AWS integration
- **Stateful:** deploy touches live infrastructure, so it needs credentials and approvals

Customizing the Infrastructure

- **Infrastructure as C#:** the AppHost is the source of truth - no hand-edited JSON manifest
- **ConfigureInfrastructure:** tweak the generated Bicep/CDK - SKUs, capacity, tags, diagnostics
- **PublishAsX methods:** decide the target shape, e.g. Container App, App Service or ECS service
- **Bring your own resources:** reference existing databases, caches or queues as external resources
- **Escape hatch:** add custom Bicep or CDK constructs when the defaults are not enough

Deployment Flow with Aspire

1. Model the distributed app in AppHost
2. Validate local behavior and telemetry in the dashboard
3. Publish artifacts from the same model
4. Bind provider resources and secrets
5. Deploy and verify health, logs, traces, and endpoints

AWS Path: ECS/Fargate Focus

- Container-first runtime with task definitions and services
- Networking and routing via VPC + load balancing patterns
- Identity and permissions through IAM roles and policies

Azure Paths: ACA, App Service, AKS

- **Azure Container Apps (ACA):** managed container runtime for microservices
- **Azure App Service:** PaaS app hosting for web/API workloads
- **Azure Kubernetes Service (AKS):** full Kubernetes control for advanced ops

Service Comparison: Compute Targets

Capability	Azure	AWS	Notes for Aspire
Serverless containers	Container Apps	ECS Fargate	Fast path for distributed APIs
Managed web app runtime	App Service	App Runner / Elastic Beanstalk	Simpler app-hosting model
Kubernetes platform	AKS	EKS	Maximum control and portability

Service Comparison: Platform Building Blocks

Concern	Azure	AWS
Container registry	Azure Container Registry	Elastic Container Registry
Secrets	Key Vault	Secrets Manager / SSM Parameter Store
Identity for workloads	Managed Identity	IAM Roles
Monitoring foundation	Azure Monitor + App Insights	CloudWatch + X-Ray
Relational database options	Azure SQL / PostgreSQL	RDS / Aurora

What is Radius?

- **Open application platform:** CNCF project for cloud-native apps across Kubernetes, Azure and AWS
- **Application graph:** app, resources and connections are modeled once, not per environment
- **Environments and recipes:** platform engineers define how a resource is provisioned
- **Separation of concerns:** developers describe needs, operators control infrastructure
- **Aspire fit:** a publish target that turns the Aspire model into a portable app definition

Radius Hosting Integration

- **Aspire.Hosting.Radius package**
 - Adds Radius as a publish/deploy environment to the AppHost
- **Model it in the AppHost**
 - `builder.AddRadiusEnvironment("prod")` – resources map to the Radius application graph
 - Aspire resources become Radius resources: containers, databases, caches, connections
- **Publish and deploy**
 - `aspire publish` generates the Bicep/Radius application definition
 - `aspire deploy` hands it to the `rad` CLI to deploy into the Radius environment
- **Why it matters**
 - Recipes decide the concrete infrastructure – the same app runs on Kubernetes, Azure or AWS
- *github.com/microsoft/aspire – `src/Aspire.Hosting.Radius`*

CI/CD and Environment Strategy

- Keep AppHost and environment overlays versioned together
- Use one pipeline template with provider-specific deployment stages
- Promote immutable images across environments
- Run smoke tests and telemetry validation after each deployment
- Keep rollback paths explicit for every cloud target

Key Takeaways

- Aspire is a delivery model, not only a local orchestrator
- One application definition can drive AWS and Azure deployments
- Cloud differences are manageable through explicit service mappings
- Start with one provider, then add the second without rewriting app architecture

Questions?

Your turn — ask anything about Aspire!

Christian Nagel

<https://www.cninnovation.com>

<https://github.com/cnilearn/bastamainz2026>



We ask for your feedback

Please vote now!

